

软件漏洞分析与挖掘

实验报告



姓 名		学 号	
班 级	软信 XXXX	指 导 教 师	刘先奇
开 设 学 期	2025-2026 秋 季 学 期		
实 验 题 目	实验一：框架（教学版）配置 实验二：活跃变量分析和迭代求解器 实验三：常量传播和 Worklist 求解器 实验四：污点分析（选做）		
实 验 日 期	2025.12.1-12.22		
评 定 成 绩		评 定 人 签 字	刘先奇
		评 定 日 期	2025.12.30

实验 1 框架（教学版）配置

一、实验目的

- 1.熟悉静态分析相关概念
- 2.学会选定的程序静态分析框架的使用方法

二、系统环境

根据个人习惯自选

三、实验工具

Tai-e 教育版

四、实验原理

Tai-e 是一个分析 Java 程序的静态程序分析框架，Tai-e 提供精炼的 IR，明晰且易拓展的接口，丰富的程序分析算法，直观易懂的框架结构，以及指导性强的文档等。Tai-e 利用 Soot 前端解析 Java 程序并帮助构建 Tai-e IR。Soot 有两个前端，分别处理 Java 源代码文件（.java）和字节码文件（.class）。其中，前者可以将源代码中的变量名保留至 IR 中，从而使得生成的 IR 更贴近源码，比后者的更易于理解。

在构建此实验环境过程中，请将 <https://github.com/pascal-lab/Tai-e-assignments> 克隆到本地，其中包含实验所需的所有依赖库等。

Tai-e 利用 Gradle 构建，并符合一般 Gradle 项目的结构，所有实验都具有如下结构：

- ① build.gradle.kts, gradlew, gradlew.bat, gradle/: Gradle 脚本和 Tai-e 项目配置文件。
- ② src/main/java: Tai-e 源代码文件夹。你需要修改该文件夹中的文件以完成实验。
- ③ src/test/java: 运行测试用例所需的测试驱动程序（test drivers）所在文件夹。
- ④ src/test/resources: 测试用例（待分析的程序）文件夹。
- ⑤ lib/: 包含 Tai-e 类的文件夹。
- ⑥ plan.yml: Tai-e 配置文件，设定了实验中需要执行的分析。
- ⑦ COPYING, COPYING.LESSER: Tai-e 许可文件。

请在实验 1 中按照 <https://tai-e.pascal-lab.net/intro/setup.html> 提供的指导，完成“配置步骤”、“以应用软件的形式运行 Tai-e”和“使用 JUnit 测试你的实验作业”3 项配置工作，

记录实验过程，以图文形式把实验过程写入下面的“五、实验步骤”小节。

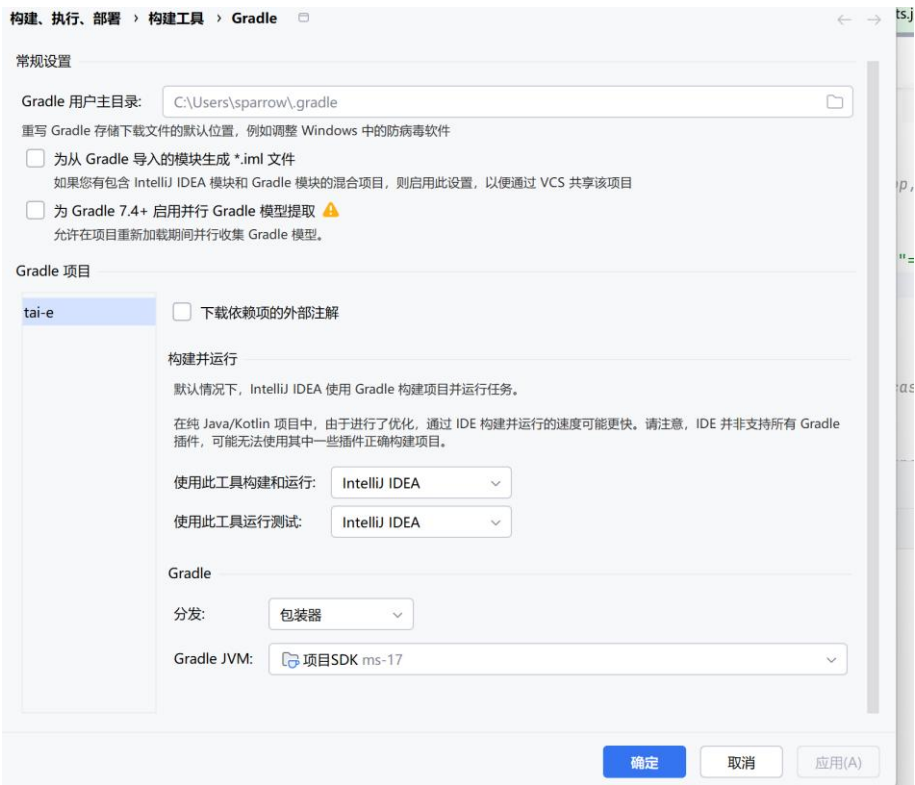
五、实验步骤

1.基础配置工作：

（1）在克隆 git 库之后，使用 IntelliJ IDEA 打开该文件后进行配置：使用 JDK 17 和 17 的语言级别

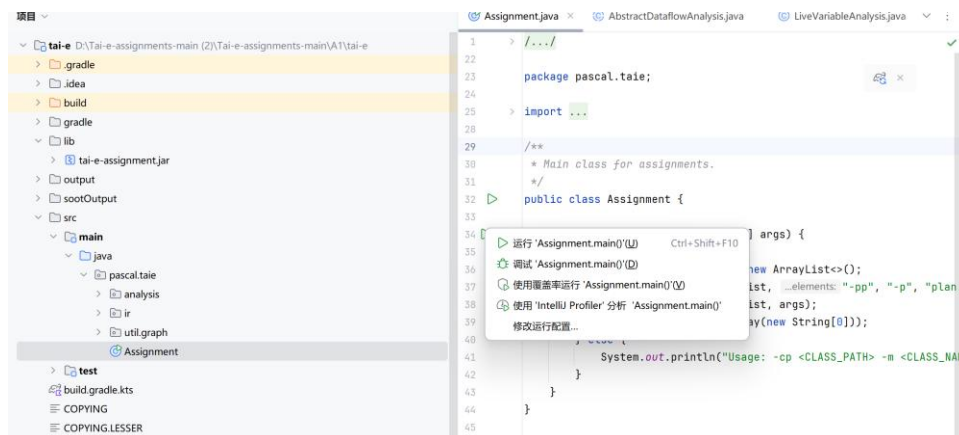


（2）在设置中将 Build and run 设置中的构建和运行工具从 Gradle 改为 IntelliJ IDEA

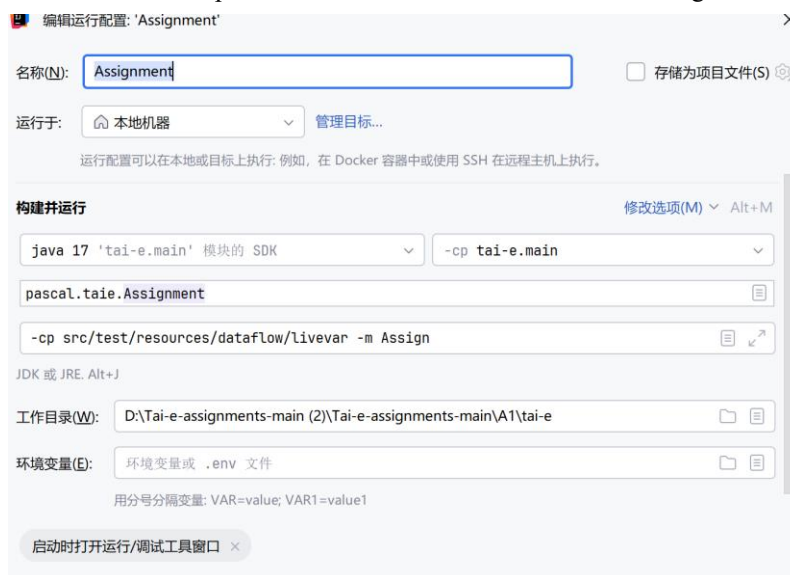


2. 以应用软件的形式运行 Tai-e:

(1) 对运行配置进行修改



(2) 配置参数 -cp src/test/resources/dataflow/livevar -m Assign



试运行结果:

```
C:\Users\sparrow\.jdk\ms-17.0.17\bin\java.exe "-javaagent:D:\IntelliJ IDEA 2025.3.1\lib\
Tai-e starts ...
Writing options to output\options.yml
WorldBuilder starts ...
Warning: main class 'Assign' does not have main(String[]) method!
10265 classes with 102565 methods in the world
WorldBuilder finishes, elapsed time: 6.69s
throw starts ...
1 classes in scope (app) of class analyses
2 methods in scope (app) of method analyses
throw finishes, elapsed time: 0.13s
cfg starts ...
cfg finishes, elapsed time: 0.06s
livevar starts ...
livevar finishes, elapsed time: 0.01s
process-result starts ...
----- <Assign: void <init>()> (livevar) -----
[0@L1] invokespecial %this.<java.lang.Object: void <init>()>(); null
[1@L1] return; null
----- <Assign: int assign(int,int,int)> (livevar) -----
[0@L4] d = a + b; null
[1@L5] b = d; null
[2@L6] c = a; null
[3@L7] return b; null
```

3. 使用 JUnit 测试你的实验作业”3 项配置工作

实验 2 活跃变量分析和迭代求解器实验目的

- 1.为 Java 实现一个活跃变量分析；
- 2.实现一个通用的迭代求解器，用于求解数据流分析问题；
- 3.通过解决该问题，进一步熟悉 Tai-e 框架，掌握静态分析基础。

一、系统环境

根据个人习惯自选

二、实验工具

Tai-e 教育版

三、实验原理

Tai-e 的框架代码中提供了包括但不限于：程序分析接口、常用数据结构（如数据流信息的表示）、必要的程序信息（如控制流图）等内容，以便在 Tai-e 上实现各类数据流分析了。

为了在 Tai-e 中实现活跃变量分析，**需要了解的类有：**

① `pascal.taie.analysis.dataflow.analysis.DataflowAnalysis`

这是一个抽象的数据流分析类，是具体的数据流分析与求解器之间的接口。也就是说，一个具体的数据流分析（如活跃变量分析）需要实现它的接口，而求解器（如迭代求解器）需要通过它的接口来求解数据流。它共有七个 API，而在实验 2 中会用到五个关键 API：分析方向、边界条件、初始条件、meet 操作、transfer 函数。可以阅读源代码与注释快速熟悉这些接口方法。这里的 transfer 函数处理的是单条语句而非程序块。

② `pascal.taie.ir.exp.Exp`

这是 Tai-e 的 IR 中的一个关键接口，用于表示程序中的所有表达式。它含有很多子类，对应各类具体的表达式。下面是一个 Exp 类继承结构的简单图示：

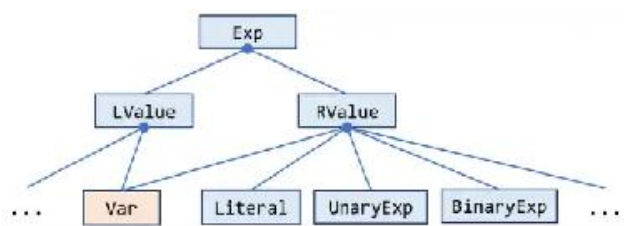


图 1. Exp 类继承结构图示

Tai-e 的 IR 把表达式分为两类: LValue 和 RValue。前者表示赋值语句左侧的表达式, 如变量 ($x = \dots$)、字段访问 ($x.f = \dots$) 或数组访问 ($x[i] = \dots$); 后者对应地表示赋值语句右侧的表达式, 如数值字面量 ($\dots = 1;$) 或二元表达式 ($\dots = a + b;$)。而有些表达式既可用于左值, 也可用于右值, 就比如变量 (用 Var 类表示)。因为本实验只进行活跃变量分析, 所以你实际上只需要关注 Var 类就足够了。

③ pascal.taie.ir.stmt.Stmt

这是 Tai-e 的 IR 中的另一个关键接口, 它用于表示程序中的所有语句。对于一个典型的程序设计语言来说, 每个表达式都属于某条特定的语句。为了实现活跃变量分析, 你需要获得某条语句中定义或使用的所有表达式中的变量。Stmt 类为这两种操作提供了对应的接口:

```
1 Optional<LValue> getDef()
2 List<RValue> getUses()
```

每条 Stmt 至多只可能定义一个变量、而可能使用零或多个变量, 因此我们使用 Optional 和 List 包装了 getDef() 和 getUses() 的结果。

④ pascal.taie.analysis.dataflow.fact.SetFact<Var>

这个泛型类用于把 data fact 组织成一个集合。它提供了各种集合操作, 如添加、删除元素, 取交集、并集等。你同样需要阅读源码和注释来理解如何使用这个类表示活跃变量分析中的各种 data fact (活跃变量分析是数据流分析的一个经典应用, 用于确定在程序的每个程序点上哪些变量是活跃的。如果一个变量在某个程序点 p 的后续路径中会被使用, 则该变量在 p 点是活跃的; 否则, 该变量在 p 点是不活跃的, 此处 data fact 可以理解为有关程序的一些实际信息, 真实信息, 这种表达源于“逻辑推理”)。

⑤ pascal.taie.analysis.dataflow.analysis.LiveVariableAnalysis

这个类通过实现 DataflowAnalysis 的接口来定义具体的活跃变量分析。

在 Tai-e 中实现迭代求解器, 你需要了解的类有:

① pascal.taie.analysis.dataflow.fact.DataflowResult

该类对象用于维护数据流分析的 CFG 中的 fact。你可以通过它的 API 获取、设置 CFG 节点的 IN facts 和 OUT facts (这里的 IN facts 和 OUT facts 可以理解为输入状态和输出状态, 当把基于程序语句语义的约束用于一个函数来表示时, 输入为 IN Facts, 则输出可以称为 OUT Facts, 详细的可参考程序分析课程)。

② pascal.taie.analysis.graph.cfg.CFG

这个类用于表示程序中方法的控制流图（control-flow graphs）。它是可迭代的，也就是说你可以通过一个 for 循环遍历其中的所有节点：

```
1 CFG<Node> cfg = ...;
2 for (Node node : cfg) {
3     ...
4 }
```

你可以通过 CFG.getPredsOf(Node) 和 CFG.getSuccsOf(Node) 这两个方法遍历 CFG 中节点的所有前驱和后继，比如你可以写：

```
1 for (Node succ : cfg.getSuccsOf(node)) {
2     ...
3 }
```

③ pascal.taie.analysis.dataflow.solver.Solver

这是数据流分析求解器的基类，包含了求解器的抽象功能。“3.2 你的任务 [重点!]”中要实现的迭代求解器就是它的一个派生类。Tai-e 会构建待分析程序的 CFG 并传给 Solver.solve(CFG)，然后世界就开始运转了。这个类中有两组 initialize 和 doSolve 方法，分别用于处理前向和后向的数据流分析。

④ pascal.taie.analysis.dataflow.solver.IterativeSolver

这个类扩展了 Solver 的功能并实现了迭代求解算法。有待同学们补全。

在认真阅读和学习上述类和相关内容的基础上，请完成（<https://tai-e.pascal-lab.net/pa1.html>）对应的“2.2 你的任务 [重点!]”和“3.2 你的任务 [重点!]”，即实现 LiveVariableAnalysis 中的 API 和实现 Solver 类。然后通过“运行与测试”介绍的方法测试你的程序。记录实验过程，以图文形式（包括可视化后的控制流图）把实验过程写入下面的“五、实验步骤”小节。

四、实验步骤

（图文形式记录实验过程和结果，此行切记在报告提交时删除。）

1.2.2 重点任务：

实验 3 常量传播和 Worklist 求解器

一、实验目的

- 1. 为 Java 实现常量传播算法；
- 2. 实现一个通用的 worklist 求解器，并用它来解决常量传播分析问题；
- 3.通过解决该问题，进一步熟悉 Tai-e 框架，掌握静态分析基础。

二、系统环境

根据个人习惯自选

三、实验工具

Tai-e 教育版

四、实验原理

实现针对 int 类型的常量传播。注意，在 Java 中，boolean、byte、char 和 short 类型在运行时实际上都以 int 值的形式进行表示和计算，因此你的分析算法也应当能处理这些类型。其他基本数据类型（例如 long、float 和 double）以及引用类型（例如 class types、array types）不在此次作业的考虑范围内，所以你可以在分析时忽略它们的值。至于语句的处理，你只需要关注等号左侧为变量且右侧只能是如下几类表达式的赋值语句：

- ① 常量，如 `x = 1`
- ② 变量，如 `x = y`
- ③ 二元运算表达式，如 `x = a + b` 和 `x = a >> b`

下表列举出了除逻辑运算符以外所有作用在 int 类型上的二元运算符。请确保你的算法能正确处理它们：

运算类型	运算符
Arithmetic	+ - * / %
Condition	== != < > <= >=
Shift	<< >> >>>
Bitwise	& ^

在 Java 中，逻辑运算符与（&&）和或（||）没有对应的字节码表示，而是以分支跳转的形式实现的。例如以下语句：

```

1  a = b || c;

```

将被转换为如下的语义等价的语句：

```

1  if (b) {
2      t = true; // t is temp variable
3  } else if (c) {
4      t = true;
5  } else {
6      t = false;
7  }
8  a = t;

```

本实验中的常量传播算法能够处理常量赋值（如 `t = true`）、变量赋值（如 `a = t`）和分支语句（如 `if(b) {…}`），所以它自然能够处理 `&&` 和 `||`。

至于等号左侧为变量、等号右侧为其它表达式的赋值语句，例如方法调用（`x = m(...)`）和字段 `load`（`x = o.f`），你需要对它们进行保守的近似处理（也许会不够精确），即把它们当作 `x = NAC(Not-A-Constant)`。对于上面没有提到的其它语句（例如字段存储 `o.f = x`），我们只需要使用恒等函数作为它们的 `transfer` 函数，此处暂时没有必要进行别名分析。

IR 相关的类（`pascal.taie.ir.*`）

这是 Tai-e 的 IR 的核心数据结构。它的每个实例储存了一个 Java 方法的各种信息，例如变量、参数、语句等等。要加深理解，可以自行阅读 API、源码实现以及注释。

实验相关的分析算法相关的类（`pascal.taie.ir.exp.Exp`）

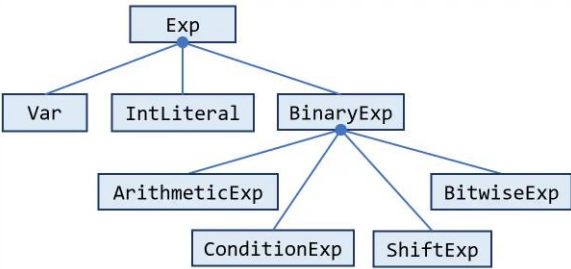


图 2. 类继承关系图

① `pascal.taie.ir.exp.Var`

这个类代表 IR 中的变量。

② `pascal.taie.ir.exp.IntLiteral`

根据 Java 的语言规范，Tai-e 中把常量称作字面量（Literals）。每个 `IntLiteral` 类的实例都表示一个程序中的整数字面量。可以通过调用 `getValue()` 方法来获取它的值。

③ `pascal.taie.ir.exp.BinaryExp`

这个类代表程序中的二元表达式。这个类的各个子类对应了表 1 中的不同种类的二元表达式，并且每个子类中都有一个内部枚举类型用于表示该类支持的运算符。例如枚举类型 `ArithmeticExp.Op` 就代表了 `ArithmeticExp`（算术表达式类）所支持的运算符，也就是 `+` `-` `*` `/` 和 `%`。

在 `Tai-e` 中，`BinaryExp` 的两个操作数都是 `Var` 类型的。这样的设计简化了分析的实现：在获取 `BinaryExp` 的操作数时，你只需要考虑它是变量的这一种可能，而不用担心它是常量或其它可能了。例如下面的语句：

```
1  x = y + 6;                                     java
```

在 `Tai-e` 中会被转化成如下的 IR：

```
1  %intconst0 = 6;      // %intconst* are temp variables introduced
2  x = y + %intconst0;  // by Tai-e to hold constant int values    java
```

④ `pascal.taie.ir.stmt.DefinitionStmt`

这是 `Stmt` 的一个子类。它表示了程序中所有的赋值语句，（即形如 `x = y` 或 `x = m(...)` 的语句）。这个类很简单。你可以通过阅读源码来决定如何使用它。

⑤ `pascal.taie.analysis.dataflow.analysis.DataflowAnalysis`

这是具体数据流分析算法需要实现的接口。它会被求解器调用。在实验中你只需要关注前 5 个 API。这些 API 会被你在后面完成的 `worklist` 求解器调用。

⑥ `pascal.taie.analysis.dataflow.analysis.constprop.Value`

这个类表示了常量分析中格上的抽象值。它的代码和注释解释了它的用法。用下列的静态方法获取格上抽象值（即该类的实例，有关格的概念请参看 <https://cs.nju.edu.cn/tiantan/software-analysis/DFA-FD.pdf#page=238> 讲解）：

`Value getNAC()`: 返回 `NAC`

`Value getUndef()`: 返回 `UNDEF`

`Value makeConstant(int)`: 返回给定整数在格上对应的抽象值

⑦ `pascal.taie.analysis.dataflow.analysis.constprop.CPFact`

这个类表示常量传播中的 `data facts`，即一个从变量（`Var`）到格上抽象值（`Value`）的映射。该类提供了各种 `map` 相关的操作，例如键值对的查询、更新等等。这些操作大多继承自 `pascal.taie.analysis.dataflow.fact.MapFact`。这些类的注释都很充分，所以你应该通过阅读源码来决定如何使用其中的 API。

⑧ `pascal.taie.analysis.dataflow.analysis.constprop.ConstantPropagation`

这个类实现了 `DataflowAnalysis`。你需要在其中编写完整的常量传播算法。

在认真阅读和学习上述类和相关内容的基础上，请完成 <https://tai-e.pascal-lab.net/pa2.html> 的“2.3 你的任务 [重点!]”和“3.2 你的任务 [重点!]”两个部分的实验要求。然后通过“运行与测试”介绍的方法测试你的程序。记录实验过程，以图文形式把实验过程写入下面的“五、实验步骤”小节。

五、实验步骤

（图文形式记录实验过程和结果，此行切记在报告提交时删除。）

实验 4 污点分析（选做）

一、实验目的

1. 为 Java 实现污点分析；
2. 强化学生自学能力，独立实现高阶静态分析技术；
3. 本实验要求学生通过自学未包含在本实验课中的其他的 Tai-e 框架实验作业，提高自己平台使用能力，深入理解静态分析的技术。

二、系统环境

根据个人习惯自选

三、实验工具

Tai-e 教育版

四、实验原理

基于基于上下文敏感的指针分析来为 Java 实现污点分析（Taint analysis）。通过实验掌握污点传播（taint transfer）的技术，这样你实现的污点分析就能够应用到生产实践中检测安全漏洞了。此外，该实验还也提供了可配置的污点分析框架，帮助使用者方便地设置污点分析中所需要的诸如 sources、sinks 等数据，考虑到前三个实验内容相对较重，所以本实验作为选做内容，希望有余力的同学参考 <https://tai-e.pascal-lab.net/pa8.html> 相关资料，通过基于污点分析的漏洞分析方法来检验自己本课程所学的相关知识掌握情况）

五、实验步骤

（图文形式记录实验过程和结果，此句在最终报告里须删除）

评分表（请在提交的报告中保留此评分表，否则会导致实验部分无法得分）

考核标准	得分
（1）正确理解和掌握实验所涉及的概念和原理（20%）；	
（2）按实验要求合理设计实验过程和步骤（40%）；	
（3）运行结果正确，实验记录准确（20%）；	
（5）实验中，学习态度和认真、踏实、具有良好的科学作风（10%）；	
（6）实验报告规范（10%）。	